RLHF — Alignment of Autoregressive LMs

GenAI (IIT-M) · source pages **128–146** · merged with the lecture transcript

Provenance. Three inputs, merged and reconciled:

- **The ink** — `GenAI-IITM.pdf` p128–146 (handwritten, vector ink, lossy text layer). Cited as p129.
- **The lecture** — `transript_RLHF.rtf`, 54 min, the professor's own words. Cited as t=21:03.
- **Derivations** — the algebra the notes gesture at but do not write out. Marked ADDED.

LECTURE stated in the transcript ERRATUM wrong in notes and/or lecture ADDED not in either source

The chapter in one sentence. You cannot supervise-train a model to be "good", because "good" is a scalar over whole sequences — so you frame the LM as a policy, take the gradient of expected reward, estimate that gradient with the policy-gradient theorem, stabilise it with importance sampling and clipping (PPO), and source the reward either from a learned Bradley–Terry preference model (PPO) or implicitly from the policy itself (DPO).

§1 Why alignment is an RL problem at all

1.1 The three paradigms t=3:39

$$\begin{aligned} &\text{supervised:} && (x, y) \sim p(x, y) && \Longrightarrow \text{ learn } p(y \mid x) \\ &\text{unsupervised:} && x \sim p(x) && \Longrightarrow \text{ learn } p(x) \\ &\text{reinforcement:} && \text{no dataset} && \Longrightarrow \text{ learn } \pi_\theta(a \mid s) \end{aligned}$$

Supervised learning *imitates a label*. RL *optimises a scalar obtained by acting*. There is no dataset of correct actions — there is an environment that returns a number.

1.2 Alignment is a training-time operation, not an inference-time one t=1:42

The professor is explicit: alignment is **not** achieved by top-*k*, temperature, or a system prompt. **The parameters themselves are changed** so that the distribution the model samples from is aligned:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{SFT}} \quad \text{becomes} \quad \theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$$

with the goal, written on p128:

$$\text{goal of alignment: make } p_\theta \text{ compatible with a desired, pre-defined rule / behaviour,}$$
$$\text{specified by human annotations or by fixed rules.}$$

1.3 Why not just supervised fine-tuning?

ADDED The notes jump straight from "goal of alignment" to "overview of RL" with no justification. The justification is two blockers:

	SFT	RLHF
Signal	a target token at every position	one scalar per *whole* completion
Granularity	dense, per-token	sparse, per-sequence
Label type	"copy this"	"A was better than B"
Capability	imitate	optimise

1. **Format blocker.** Preference data is *comparative*, not generative. A human ranks two answers reliably; they cannot *write* the ideal answer. There is no target string, so there is nothing to put through cross-entropy.
2. **Locality blocker.** "Helpful" is a property of the *whole sequence* — a derivation is correct or not, a refusal is appropriate or not. You cannot assign a per-token label to that. The signal lives at the end.

So you need an objective on complete sequences, optimised in expectation over your own sampling distribution. That *is* an MDP with a policy. Hence RL.

1.4 Verifiable rewards — the load-bearing idea

ADDED **Nothing on p128–146 mentions this. It is the most useful idea in the first five minutes of the lecture.** t=2:02–3:14

$$\underbrace{r:\ S\times A\to\mathbb{R}}_{\text{verifiable}}\quad:\quad r\text{ is deterministic and checkable}\quad\text{vs.}\quad\underbrace{r\text{ is a judgement, not a check}}_{\text{non-verifiable}}$$

- **Math and code are where RLHF works best** t=2:37 — the reward is a test-case outcome, essentially binary: $r(s_T, a_T) \in \{0, 1\}$, cheap, noiseless, unlimited supply.
- **Chat is the hard case** t=3:06 — the reward is "vague". You cannot verify politeness, so you must *learn* r_ϕ from preferences, and every error in it becomes an error in your policy.

> **The spine of the whole chapter** t=32:36:
> "your policy gradient algorithm is as good as your reward model."

1.5 The monster picture t=35:20

```
pre-train on the whole internet   ->  ugly monster   (unsupervised / pre-training)
        + supervised fine-tuning  ->  pink face     (still monstrous)
        + RLHF                    ->  smiley face
```

Alignment is a **thin veneer over a pre-trained distribution nobody chose**. It is also expensive: millions of dollars just to make the model say "thank you" t=36:09 — because the advantage function needs rewards, and rewards need human labels.

§2 The Markov Decision Process p129, t=7:27

symbol	name	form
$\mathcal{S}$	state space	$\{s_1, \dots, s_M\}$ — finite, for this course
$\mathcal{A}$	action space	$\{a_1, \dots, a_n\}$
p	transition kernel	$p(s_{t+1} \mid s_t, a_t)$ — first-order Markov ⇒ "MDP"
r	reward	$r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, scalar, may be negative
γ	discount factor	$\gamma \in (0, 1)$
ρ_0	initial state distribution	$\rho_0(s_0)$
π_θ	policy	$\pi_\theta(a_t \mid s_t)$, parameterised by θ

> **"MDP" explained** t=9:38 — "it's called Markov because the transition kernel is *first-order* Markov by definition." The next state depends only on the current state and action, never on history: $p(s_{t+1} \mid s_t, a_t, \dots, s_0) = p(s_{t+1} \mid s_t, a_t)$.

> **Agent ≡ policy** t=13:27 — "if there is an agent there is a policy, because that's what defines what an agent is." The two words are interchangeable for the rest of the chapter.

2.1 What γ is actually for

The notes introduce γ with no justification at all. The lecture's answer t=14:02: γ^t controls **how much weight distant reward gets**. $\gamma \to 0 \to$ only the immediate reward matters; $\gamma \to 1 \to$ all reward equally weighted, and the infinite sum may diverge.

> **ERRATUM** The lecture's phrasing — "if you want the *recent* rewards to have the highest impact, scale γ accordingly" — is loose. Mechanically γ^t **decays with** t, so reward far in the future is squashed. The right word is **discounting the future**, not "recency". Say it that way and you never confuse yourself.

§3 Trajectory, return, objective p130, t=10:04

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$$

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t)$$

Three factors, and **exactly one of them contains** θ — the middle one. Hold onto this; it is the engine of §4.

$$R(\tau) = \sum_{t=0}^{T} \gamma^t r(s_t, a_t) \qquad \text{(discounted return)}$$

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\big[R(\tau)\big] = \int p_\theta(\tau)\, R(\tau)\, d\tau, \qquad \theta^\star = \arg\max_\theta J(\theta)$$

"It's just the chain rule of probability" t=13:04. Start in accordance with ρ_0, then for each t multiply the policy probability of the action by the kernel probability of the next state.

All RL methods solve *this same problem* t=15:24: return the parameters maximising expected discounted return over all trajectories. The chapter only uses gradient methods, because the policy is a neural network and we already know how to backprop through it t=16:08.

§4 The Policy Gradient Theorem p131–132, t=17:52

4.1 The obstruction

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau = \int \underbrace{\nabla_\theta p_\theta(\tau)}_{\text{the measure}} R(\tau)\, d\tau$$

Look at where θ sits. It is **not** in the reward — it is in the *measure*. You are not differentiating a function of θ; you are differentiating a distribution. And since $\nabla_\theta R(\tau) = 0$, $\mathbb{E}[\nabla_\theta R(\tau)] = 0$ identically and uselessly. The integral is inaccessible twice over: p_θ is unknown, and the trajectory space $|\mathcal{V}|^T$ cannot be enumerated.

t=18:15 "Note that we don't have this distribution p_θ with us — the same problem that we are encountering all over the course." Exactly the shape of the VAE's intractable integral. Same fix: get θ inside a log, then Monte-Carlo it.

4.2 The log-derivative trick t=18:44

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau) \qquad \left(\text{since } p \cdot \tfrac{1}{p}\nabla p = \nabla p\right)$$

$$\nabla_\theta J(\theta) = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau = \mathbb{E}_{\tau\sim p_\theta}\Big[\nabla_\theta \log p_\theta(\tau)\, R(\tau)\Big]$$

Now $\nabla_\theta \log p_\theta(\tau)$ is computable — and the expectation can be estimated by sample averaging over rollouts.

4.3 The three factors collapse to one

$$\log p_\theta(\tau) = \underbrace{\log \rho_0(s_0)}_{\theta\text{-free}} + \sum_t \underbrace{\log \pi_\theta(a_t \mid s_t)}_{\text{only term with } \theta} + \sum_t \underbrace{\log p(s_{t+1} \mid s_t, a_t)}_{\theta\text{-free}}$$

$$\Longrightarrow \quad \nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

ρ_0 and the kernel differentiate to zero. They simply **cancel away** t=20:25.

> **The result** p131:
>
> $$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\Big[\Big(\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big) R(\tau)\Big]$$

4.4 Why this is the "very very strong result" t=21:03

> **You do not need the state-transition distribution.** "As long as you know how the agent behaves and you have access to the reward function, you can train the policy *without knowing what the transition probabilities are*." The gradient contains no $p(s_{t+1} \mid s_t, a_t)$ term at all.

That is the whole point. In an LLM the "environment" is the world, and its transition kernel is **not written down anywhere** — there is no model of how the world responds to a token. The theorem says you never need one.

Every method in the rest of the chapter — REINFORCE, PPO, TRPO, DPO, GRPO — is a modification of this one equation t=21:49.

§5 Variance reduction: baseline, value, advantage p132–134, t=23:49

5.1 The estimator is high-variance

$\nabla_\theta J$ is now an expectation, so it will be approximated by a sample average — and as soon as you have an estimator you have bias and variance t=23:25. Empirically the plain estimator has a lot of variance t=24:39. The fix: subtract a baseline $b(s_t)$ from the return.

$$R_t = \sum_{k\geq t} \gamma^{k-t} r(s_k, a_k) \qquad \text{(reward-to-go, p132)}$$

$$R_t \;\longmapsto\; R_t - b(s_t)$$

The notes shift the time index from t to k on p132 and never say why t=24:02: it is so that R_0 is the full return and R_t is the return from time t onward.

5.2 Why a baseline is free

$$\mathbb{E}_{a_t\sim\pi}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, b(s_t)\big] = b(s_t)\sum_a \nabla_\theta \pi_\theta(a \mid s_t) = b(s_t)\,\nabla_\theta \underbrace{\sum_a \pi_\theta(a \mid s_t)}_{=1} = b(s_t)\cdot 0 = 0$$

So subtracting any $b(s_t)$ that does not depend on the action leaves the gradient **exactly unbiased**. It changes the estimator's variance only.

> **ERRATUM — lecture** t=24:57 "Suppose you take an estimator that has high variance. You subtract a constant — the variance of the estimator would simply change." **This is false for a constant.**
>
> $$\mathrm{Var}(X - c) = \mathrm{Var}(X) \quad \text{for any constant } c \qquad \text{but} \qquad \mathrm{Var}(X - b) = \mathrm{Var}(X) - 2\,\mathrm{Cov}(X, b) + \mathrm{Var}(b)$$
>
> Subtracting a constant does **nothing**. Variance falls only because $b(s)$ is *correlated with the return* and varies across states — you are removing the between-state variance. A state-dependent baseline helps; a constant one is a no-op.

5.3 The standard choice: the value function p132–133, t=25:44

$$V^\pi(s) = \mathbb{E}_\pi\big[R_t \mid s_t = s\big], \qquad Q^\pi(s,a) = \mathbb{E}_\pi\big[R_t \mid s_t = s,\ a_t = a\big]$$

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

> **Intuition, in the professor's words** t=28:29: the Q function is the accumulated reward given that you started at s *and took this particular action*; the value function is the accumulated reward given only that you started at s — "I don't care what the action is." The difference asks: **"does taking this action lead to a better reward or not?"** Hence the name *advantage*.

5.4 $R_t - V(s_t)$ is an unbiased estimate of the advantage t=30:01

$$\begin{aligned}\mathbb{E}\big[R_t - V^\pi(s_t) \mid s_t{=}s,\ a_t{=}a\big] &= \underbrace{\mathbb{E}\big[R_t \mid s_t{=}s,\ a_t{=}a\big]}_{Q^\pi(s,a)} - V^\pi(s) \quad \text{(linearity; } V^\pi(s) \text{ is constant given } s\text{)} \\ &= Q^\pi(s,a) - V^\pi(s) = A^\pi(s,a) \quad \text{(definitions)}\end{aligned}$$

Therefore $R_t - V(s_t)$ is a **single-sample estimator** of the advantage: drop the expectation from an unbiased estimator and what remains is a one-sample estimate of it t=31:02.

5.5 The key equation of every RLHF paper p134, t=31:37

> $$\nabla_\theta J(\theta) \;\approx\; \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\; hat{A}^\pi(s_t, a_t)\Big]$$
>
> "This equation is the key equation that you see in all RLHF papers."

Note the $\approx$: it is not *equal* to the advantage, it is a single-sample estimate of it. The professor is careful about this t=31:37.

§6 An autoregressive LM as an RL policy p134–135, t=34:29

$$\begin{aligned} s_0 &= \text{prompt} = x_{<1} && \leftarrow \rho_0 \text{ is the prompt distribution} \\ a_0 &= x_1 && \text{first output token} \\ s_1 &= [\,s_0, a_0\,], \quad a_1 = x_2 && \text{concatenate, then predict} \\ &\vdots \\ \mathcal{A} &= \mathcal{V}, \qquad \pi_\theta(a_t \mid s_t) = \mathrm{softmax}(z_t) \in \Delta^{|\mathcal{V}|} \end{aligned}$$

Three consequences, all stated in the lecture:

1. **The LM is the policy** t=37:07 — "the transformer that gives you a distribution over the tokens of the vocabulary *is* taken as a policy." No separate head, no wrapper.
2. **The action space is the vocabulary**, so $|\mathcal{A}| \approx 5\times10^4 - 2\times10^5$ — enormous by RL standards. And a_t is a *token id*, a one-hot over $\mathcal{V}$, not the token's embedding.
3. **Updating the policy = changing the transformer's weights** t=38:45. "Training a language model using RL is equivalent to changing the parameters of the language model itself."

ADDED **Where is r nonzero?** Neither source says it plainly, and it matters.

$$r(s_t, a_t) = 0 \;\; \forall\, t < T, \qquad r(s_T, a_T) = r_\phi(x, y) \;\in\; \mathbb{R}$$

The reward arrives **once, at the end of the episode**, because a preference is a judgement about a complete response. This is why the advantage estimate in LLM-PPO is dominated by "how did the episode end" and why the value head exists at all (§8). Episodes terminate at EOS or max length, so in practice $\gamma = 1$ — a quiet mismatch with the infinite-horizon discounted MDP of §2.

§7 PPO, part 1: importance sampling and clipping p136–140, t=39:10–52:42

7.1 The on-policy problem p136, t=41:18

$$\theta_{t+1} = \theta_t + \alpha\, \nabla_\theta J(\theta_t), \qquad \nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t\Big]$$

The expectation is over trajectories **from the very policy being optimised**. That is what **on-policy** means: every update needs fresh rollouts, because the moment θ moves, the sample you just collected is from the wrong distribution.

For an LLM a rollout means running inference on the model you are training — generating full responses, several per prompt. Expensive. You want to **reuse** them.

7.2 Importance sampling p137, t=42:34

You want an expectation under p but only have samples from q:

$$\mathbb{E}_{x\sim p}\big[f(x)\big] = \int p(x) f(x)\, dx = \int q(x) \underbrace{\frac{p(x)}{q(x)}}_{\text{importance weight}} f(x)\, dx = \mathbb{E}_{x\sim q}\Big[\frac{p(x)}{q(x)} f(x)\Big]$$

Price paid: you must be able to **evaluate the density ratio** at every point. The professor's framing t=44:39: the advantage is you can estimate an expectation without samples from the target; **the cost is the ratio**.

7.3 Why a transformer is the perfect place to pay that price t=45:48

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

You can compute the **log-likelihood a token got under the old checkpoint** (it was written to the rollout buffer) and under the current one (one forward pass). The ratio is two numbers and a subtraction. That is only possible because the policy is an *explicit likelihood model* — you could never do this with a Q-learning agent.

7.4 The surrogate objective p138, t=46:48

Two forms circulate. They are **not the same function** — the lecture is explicit about this t=47:28:

$$\underbrace{\mathbb{E}_{\pi_{\text{old}}}\big[r_t(\theta)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\big]}_{\text{policy gradient}} \quad \text{vs.} \quad \underbrace{\mathbb{E}_{\pi_{\text{old}}}\big[r_t(\theta)\, \hat{A}_t\big]}_{\text{surrogate } L(\theta)}$$

They differ by *ratio times grad-log* versus *grad-ratio*. The bridge:

$$\nabla_\theta r_t(\theta) = \nabla_\theta \frac{\pi_\theta}{\pi_{\text{old}}} = \frac{\pi_\theta}{\pi_{\text{old}}} \nabla_\theta \log \pi_\theta = r_t(\theta)\, \nabla_\theta \log \pi_\theta$$

$$\nabla_\theta L(\theta)\big|_{\theta=\theta_{\text{old}}} = \nabla_\theta J(\theta)\big|_{\theta=\theta_{\text{old}}}$$

"Take this as homework and do it" t=48:00. Note the qualifier: the surrogacy holds **at** θ_{old}, i.e. it is a first-order approximation to J around the current policy — not an identity everywhere.

ADDED In an LLM implementation the ratio is **per-token**, r_t for the single token being scored, not the product over the whole trajectory. The trajectory-level product telescopes to $\pi_\theta(y \mid x)/\pi_{\text{old}}(y \mid x)$ and is dominated by the first tokens, which destroys credit assignment. Every real PPO-for-LLM clips per token.

7.5 Off-policy, and why it is *better* here t=46:16

With the ratio, rollouts come from π_{old}, not π_θ. This is the **off-policy** version. The lecture's claim that off-policy is preferred is about **stability**: taking a gradient from a rollout of the *current* policy at that exact instant is unstable; taking several gradient steps against a slightly stale policy, reweighted, is smoother and cheaper per sample.

7.6 The failure mode p139, t=49:22

$$\pi_\theta \text{ far from } \pi_{\text{old}} \implies r_t(\theta) \text{ explodes} \implies L(\theta) \text{ explodes} \implies \text{high-variance, unstable updates}$$

7.7 TRPO: a trust region p140, t=50:04

$$\max_\theta \ \mathbb{E}_{\pi_{\text{old}}}\big[r_t(\theta)\hat{A}_t\big] \quad \text{s.t.} \quad \mathbb{E}_{s\sim\pi_{\text{old}}}\Big[D_{\text{KL}}\big(\pi_{\text{old}}(\cdot \mid s) \,\big\|\, \pi_\theta(\cdot \mid s)\big)\Big] \le \delta$$

δ is a hyperparameter and the region it defines is the **trust region** — the set of policies close enough to the old one that the surrogate is still trustworthy.

> **Problem** t=51:53: **solving this constrained problem is non-trivial** — the KL is expensive at scale and constrained optimisation wants a second-order solver. So PPO replaces the constraint with a cheap *surrogate for* it.

7.8 PPO: clip instead of constrain p140, t=52:05

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \ \text{clip}\big(r_t(\theta),\,1-\epsilon,\,1+\epsilon\big)\hat{A}_t\Big)\Big], \qquad \epsilon \approx 0.1\text{–}0.2$$

Read the $\min$ as a **pessimistic bound**. Two regimes:

$\hat{A}_t$	unclipped term wants	what clip does	result
> 0	r as large as possible (raise the good action's prob)	caps the gain at $r = 1+\epsilon$	no incentive past $1+\epsilon$
< 0	r as small as possible (lower the bad action's prob)	caps the gain at $r = 1-\epsilon$	no incentive below $1-\epsilon$

$\min$ picks the *worse* of the two, so the objective can never be gamed by a runaway ratio. A ratio stuck inside $[1-\epsilon, 1+\epsilon]$ is, approximately, a bound on the KL t=52:17 — a cheap surrogate for the trust region.

> **Lineage** t=50:43, worth memorising as one chain:
>
> ```
> REINFORCE  -- no importance sampling, on-policy, plain grad J
>   -> PPO  -- importance sampling, off-policy, surrogate L = r * A
>   -> TRPO -- PPO + explicit KL constraint (trust region)
>   -> PPO  -- TRPO's constraint replaced by clipping
>   -> GRPO -- a TRPO-family variant whose advantage needs no reward model
> ```

§8 The full PPO loss p141, t=52:42

$$L^{\text{PPO}}(\theta) = \underbrace{L^{\text{CLIP}}(\theta)}_{\text{policy}} - \underbrace{c_1\big(V_\theta(s_t) - \hat{R}_t\big)^2}_{\text{value regression}} + \underbrace{c_2\,\mathcal{H}\big(\pi_\theta(\cdot \mid s_t)\big)}_{\text{entropy bonus}}, \qquad \theta^\star = \arg\max_\theta L^{\text{PPO}}(\theta)$$

term	what it is	what it is for
L^{CLIP}	clipped policy-gradient surrogate	raise the probability of high-advantage actions
$\big(V_\theta(s) - \hat{R}\big)^2$	a second head on the network, trained by regression onto the observed return	you need V to compute $A = Q - V$; this trains it
$\mathcal{H}(\pi_\theta)$	entropy of the policy	keep the policy stochastic; stop premature collapse onto one response

> **ERRATUM — notes** Two problems with p141 as written:
>
> 1. **Signs are garbled** in the ink. Since the notes write $\theta^\star = \arg\max$, the value term must be **subtracted** (you *minimise* the squared error) and the entropy **added** (you *maximise* entropy). Written as a loss to *minimise*, all signs flip: $-L^{\text{CLIP}} + c_1\,\text{MSE} - c_2\,\mathcal{H}$. Pick one convention and stay in it.
> 2. **A term is missing that makes RLHF-PPO what it is.** The notes never include the **KL penalty to a frozen reference policy** — yet p144 relies on exactly that term when motivating DPO. In RLHF the per-token reward is shaped as
>
> $$r_t = -\beta \log \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{ref}}(a_t \mid s_t)}$$
>
> plus the terminal $r_\phi(x, y)$. Without it, PPO will happily drift into degenerate text that games the reward model (§9.4).

ADDED **Advantage estimation in practice.** Neither source says how $\hat{A}$ is actually computed. The standard is **GAE** (generalised advantage estimation), a geometric interpolation between one-step TD error and the full Monte-Carlo return:

$$\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t), \qquad \hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \, \delta_{t+l}, \qquad \lambda \approx 0.95$$

$\lambda \to 0$: low variance, high bias (trusts V). $\lambda \to 1$: high variance, unbiased (trusts the observed return).

§9 Reward modelling p141–143, t=53:07

9.1 The naive route, and why it fails p142

In principle one could regress $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ on $\big[(s,a),\, r\big]$ tuples by ERM. "However, obtaining such data is not easy." The lecture's phrasing t=53:32: absolute reward labels are expensive, noisy, and inconsistent across annotators — two people give the same response 6/10 and 9/10.

So switch the **data format**: ask for *comparisons* instead of scores. Humans are far more reliable at "which of these two is better" than at "rate this 0–10".

$$\mathcal{D} = \Big\{\big(x,\, y_w,\, y_l\big)\Big\}, \qquad x = \text{prompt}, \quad y_w = \text{preferred}, \quad y_l = \text{non-preferred}$$

The notes end p142 with the live question that §9.2 answers: **"Given preferential data, how would one build a reward model?"**

9.2 The Bradley–Terry model p143

$$P(y_w \succ y_l \mid x) = \frac{\exp\big(r_\phi(x, y_w)\big)}{\exp\big(r_\phi(x, y_w)\big) + \exp\big(r_\phi(x, y_l)\big)} = \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)$$

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Fit by maximum likelihood on the preference labels:

$$\mathcal{L}_{\text{RM}}(\phi) = \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}\Big[\log \sigma\Big(r_\phi(x, y_w) - r_\phi(x, y_l)\Big)\Big], \qquad \phi^\star = \arg\max_\phi \mathcal{L}_{\text{RM}}(\phi)$$

The two lines on p143 are the same object: dividing numerator and denominator by $e^{r_\phi(x, y_w)}$ turns the softmax form into the sigmoid-of-difference form. Both appear in the notes; do not be confused by the duplication.

9.3 What σ is doing, mechanically

$$r_\phi(x, y_w) - r_\phi(x, y_l) \to \begin{cases} +\infty & P(y_w \succ y_l) \to 1 \quad \text{confidently right} \\ 0 & P(y_w \succ y_l) \to \tfrac{1}{2} \quad \text{cannot tell them apart} \\ -\infty & P(y_w \succ y_l) \to 0 \quad \text{confidently } \textit{wrong} \end{cases}$$

So the loss is a logistic loss on the **reward difference**. The gradient pushes $r_\phi(x, y_w)$ up and $r_\phi(x, y_l)$ down with weight $\sigma\big(r_\phi(x, y_l) - r_\phi(x, y_w)\big)$ — how wrong the model currently is. A pair the model already gets right contributes almost nothing.

ERRATUM — both sources Three properties of Bradley–Terry that neither the notes nor the lecture state, and that a practitioner needs:

1. **Only differences are identifiable.**

   $$r_\phi \longmapsto r_\phi + c \implies \mathcal{L}_{\text{RM}} \text{ unchanged} \quad \forall c \in \mathbb{R}$$

   So the *absolute scale and offset of the reward are meaningless* — only gaps matter. This is not a technicality: it is exactly the freedom DPO exploits in §10.3 to cancel $Z(x)$.
2. **Ties are unrepresentable.** BT demands a strict preference. Real annotation has ties and near-ties; you either drop them or model them (Rao–Kupper). Dropping them biases the reward toward whichever side won the coin flip.
3. **BT assumes the preference is stochastic with a logistic link on a scalar difference.** It is a modelling assumption, not a theorem. If human preference depends on features that no scalar r captures, the reward model is misspecified and PPO optimises the misspecification.

9.4 Reward hacking — the failure mode that shapes RLHF

ADDED r_ϕ is a **proxy** trained on finite data. PPO is an optimiser with no other objective. Give it enough steps and it will find inputs where r_ϕ is high and human preference is not — sycophancy, excessive length, confident-sounding nonsense. This is **Goodhart's law with a gradient**. The mitigation is the KL leash:

$$\max_\theta \; \mathbb{E}_{x,\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta(\cdot\mid x)\,\big\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)$$

Which is the term missing from p141 (§8). β trades reward against faithfulness. **This exact objective is what DPO will solve in closed form** — the next section is one algebraic step away.

§10 Direct Preference Optimisation (DPO) p144–146

10.1 The goal p144

$$\text{optimise } \pi_\theta \text{ on preference data } \mathcal{D} \quad \text{without a reward model}$$

Start from the constrained objective of §9.4 — the notes' line $L_{\text{policy}} = \mathbb{E}[r] - \beta\, D_{\mathrm{KL}}$:

$$\max_\theta \; \mathcal{L}_{\text{policy}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r(x,y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta(\cdot\mid x)\,\big\|\,\pi_{\mathrm{ref}}(\cdot\mid x)\big)$$

10.2 It has a closed-form solution p144

$$\pi_r(y\mid x) = \frac{1}{Z(x)}\,\pi_{\mathrm{ref}}(y\mid x)\exp\Big(\frac{1}{\beta}\,r(x,y)\Big), \qquad Z(x) = \sum_y \pi_{\mathrm{ref}}(y\mid x)\exp\Big(\frac{1}{\beta}\,r(x,y)\Big)$$

ADDED **Why this is the solution.** The objective separates over x, so fix x and maximise over the distribution $\pi(\cdot\mid x)$ pointwise, subject to $\sum_y \pi(y\mid x) = 1$:

$$\begin{aligned}
\mathcal{L} &= \sum_y \pi(y)\Big[r(y) - \beta\log\frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}\Big] + \mu\Big(\sum_y \pi(y) - 1\Big) \\
\frac{\partial\mathcal{L}}{\partial\pi(y)} &= r(y) - \beta\log\frac{\pi(y)}{\pi_{\mathrm{ref}}(y)} - \beta + \mu = 0 \\
\Longrightarrow\quad \pi(y) &\;\propto\; \pi_{\mathrm{ref}}(y)\exp\big(r(y)/\beta\big) \qquad \underbrace{\exp\big((\mu-\beta)/\beta\big)}_{\text{constant in } y \;\Rightarrow\; \text{absorbed into } Z(x)}
\end{aligned}$$

The KL term is precisely what makes this tractable — without it the optimum is the degenerate "put all mass on $\arg\max r$".

But $Z(x)$ is intractable — it is a sum over every possible response. That is the obstruction the notes name on p144, and why you cannot just sample from π_r and be done.

10.3 Invert the closed form p145

Take logs and rearrange — $Z(x)$ does not depend on y, so it is a constant you can isolate:

$$\begin{aligned}
\log\pi_r(y\mid x) &= \log\pi_{\mathrm{ref}}(y\mid x) - \log Z(x) + \tfrac{1}{\beta}r(x,y) \\
\Longrightarrow\quad r(x,y) &= \beta\log\frac{\pi_r(y\mid x)}{\pi_{\mathrm{ref}}(y\mid x)} + \beta\log Z(x)
\end{aligned}$$

This line is the pivot of the whole algorithm. It says the optimal reward *is* the log-ratio of the optimal policy to the reference policy, plus a per-prompt constant. So the reward is not something you need to learn separately — **it is implicitly present in the policy itself.**

10.4 Substitute into Bradley–Terry — $Z(x)$ cancels p145–146

$$\begin{aligned}
P(y_w \succ y_l\mid x) &= \sigma\Big(r(x,y_w) - r(x,y_l)\Big) \\
&= \sigma\Big(\beta\log\frac{\pi_r(y_w\mid x)}{\pi_{\mathrm{ref}}(y_w\mid x)}\underbrace{+\beta\log Z(x)}_{}\; - \beta\log\frac{\pi_r(y_l\mid x)}{\pi_{\mathrm{ref}}(y_l\mid x)}\underbrace{-\beta\log Z(x)}_{\text{cancels}}\Big) \\
&= \sigma\Big(\beta\log\frac{\pi_r(y_w\mid x)}{\pi_{\mathrm{ref}}(y_w\mid x)} - \beta\log\frac{\pi_r(y_l\mid x)}{\pi_{\mathrm{ref}}(y_l\mid x)}\Big)
\end{aligned}$$

Why the cancellation works: $Z(x)$ is a function of x **only** — and y_w, y_l share the **same prompt** x. So it appears twice with the same value and subtracts out. **That is the entire trick of DPO.** If the two responses had come from different prompts, DPO would not exist.

Now replace π_r with the policy you are actually training, π_θ, take the negative log-likelihood, done:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

p146 — the notes' "objective for DPO".

The notes close p146 with: "*The above expression is independent of the reward r_ϕ.*" Correct — and that is the headline. No reward model, no PPO, no rollouts, no value head. Just preference pairs and a log-ratio.

10.5 Reading the loss

$$\hat{r}_\theta(x,y) \triangleq \beta\log\frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \qquad (\text{the } \textit{implicit reward})$$

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log\sigma\left(\hat{r}_\theta(x,y_w) - \hat{r}_\theta(x,y_l)\right)\right]$$

It is **formally identical to the Bradley–Terry reward-model loss of §9.2** — the same $-\log\sigma(\text{gap})$ — with r_ϕ replaced by $\hat{r}_\theta$. That is the entire content of DPO:

DPO is Bradley–Terry reward modelling with the policy as its own reward function. The reward is not absent. It is *implicit*, and it is $\beta\log(\pi_\theta/\pi_{\text{ref}})$.

$$\nabla_\theta\mathcal{L}_{\text{DPO}} = -\beta\,\mathbb{E}\Big[\sigma\big(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w)\big)\Big(\nabla_\theta\log\pi_\theta(y_w \mid x) - \nabla_\theta\log\pi_\theta(y_l \mid x)\Big)\Big]$$

Two readings worth keeping:

- **Weight = surprise.** $\sigma(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w))$ is large when the model currently prefers the *loser*. Pairs the model already ranks correctly get almost no gradient. Same structure as §9.3.
- **Push and pull.** Raise the log-likelihood of y_w, lower that of y_l — but both are *relative to* π_{ref}, so the model is only ever asked to **change its mind about which of two responses is better**, not to relearn language from scratch.

10.6 PPO vs DPO

	PPO / RLHF	DPO
Reward	explicit r_ϕ, trained separately	implicit $\beta\log(\pi_\theta/\pi_{\text{ref}})$
Data	on-policy rollouts, generated live	offline preference pairs (x,y_w,y_l)
Models in memory	4 — policy, ref, reward, value	2 — policy, ref
Cost per step	high (generation dominates)	one forward+backward per pair
Failure mode	reward hacking (§9.4)	likelihood displacement / over-optimisation on the offline set
Theory status	policy gradient, unbiased-ish, stable-ish	exact MLE under the BT model *if* the reference is the true optimum

ERRATUM — caveat on "without the reward model" DPO's derivation assumes π_{ref} is the policy that *would* be optimal under some reward, so that $r = \beta\log(\pi^\star/\pi_{\text{ref}}) + \beta\log Z$ holds exactly. In reality π_{ref} is just your SFT checkpoint. And the loss only ever sees y_w, y_l as **strings** — so it can lower $\log\pi_\theta(y_l \mid x)$ by pushing mass onto responses that are *neither* ("likelihood displacement"). DPO is simpler, not strictly better.

§11 Error ledger

where	what is written	what is true
p130	"policy updating using grad **descent**" next to $\theta \leftarrow \theta + \alpha\nabla_\theta J$	The plus sign means **ascent**. Words wrong, formula right. The lecture self-corrects: "gradient descent or gradient ascent rather" t=22:52.
p130	$R(\tau) = \sum_t r(s_t,a_t)$	γ was declared on p129, so the return must be $\sum_t \gamma^t r(s_t,a_t)$. The lecture writes the discounted form t=14:08; the ink does not.
t=24:57	"subtract a **constant** … the variance would simply change"	**False.** $\text{Var}(X-c) = \text{Var}(X)$. Variance falls only for a *state-dependent* baseline $b(s)$ correlated with the return.
t=14:02	γ makes "**recent** rewards" matter most	Mechanically γ^t **discounts the future**. Recency is the wrong word; the mechanism is exponential decay in t.

p132	baseline $b(s_t)$ reduces variance — no condition given	Reduces variance **only** if $\mathrm{Cov}(X, b) > 0$. And unbiasedness requires b to not depend on a_t. Neither caveat is stated.
p141	full PPO loss has no reference-policy KL	RLHF-PPO needs $-\beta D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$, or it reward-hacks. p144 then silently assumes this term exists when motivating DPO. Internal inconsistency.
p141	signs of the value and entropy terms are unclear in the ink	With $\arg\max$: $-c_1\mathrm{MSE} + c_2\mathcal{H}$. With a minimised loss: all signs flip.
p143	BT gives the likelihood, nothing else	Only **differences** are identifiable ($r \mapsto r + c$ is a symmetry); ties are unmodellable; the logistic link is an *assumption*. All three matter downstream.
p128–146	no mention of verifiable rewards	The lecture's central practical point t=2:02: RLHF works best when r is checkable (math, code). Chat is the hard case.
p135	state/action mapping drawn, reward timing never stated	The reward is **terminal only** — one scalar per response. It governs everything about the advantage estimate.

§12 The whole chapter in one table

layer	what it adds	the object
0 why	the premise	MLE matches a corpus; "good" is a scalar over whole sequences; preference data is comparative, so there is no per-token target ⇒ frame the LM as a policy.
1 MDP p129–130	defines the game	$p_\theta(\tau) = \rho_0(s_0)\prod_t \pi_\theta(a_t \mid s_t)p(s_{t+1} \mid s_t, a_t)$ $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$
2 gradient p131–134	makes it computable	$\nabla_\theta J = \mathbb{E}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\Big]$ kernel-free — the payoff
3 stability p136–141	makes it reusable	$r_t = \frac{\pi_\theta}{\pi_{\mathrm{old}}}$, $L = \mathbb{E}_{\pi_{\mathrm{old}}}[r_t\hat{A}_t]$, $L^{\mathrm{CLIP}} = \mathbb{E}\big[\min(r_t\hat{A}_t, \mathrm{clip}(r_t, 1\pm\epsilon)\hat{A}_t)\big]$
4 reward p141–143	supplies r	$P(y_w \succ y_l) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$ r_ϕ is a proxy ⇒ Goodhart ⇒ KL leash to π_{ref}
5 DPO p144–146	deletes the reward model	$r = \beta\log\frac{\pi^\star}{\pi_{\mathrm{ref}}} + \beta\log Z(x)$, Z cancels in $r_w - r_l$ $\mathcal{L}_{\mathrm{DPO}} = -\mathbb{E}\big[\log\sigma\big(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\big)\big]$

§13 Glossary — every symbol in the chapter

symbol	meaning	where / trap
$\mathcal{S}$, $\mathcal{A}$	state space, action space	for an LLM, $\mathcal{A} = \mathcal{V}$ (the vocabulary)
$p(s' \mid s, a)$	transition kernel	never needed by the policy-gradient theorem — that is the point
$r(s, a)$	reward, a scalar	may be negative; for LLMs it is terminal-only
γ	discount factor $\in (0, 1)$	discounts the *future*, not "favours recency" (§2.1)
$\rho_0(s_0)$	initial state distribution	for an LLM this is the **prompt distribution**
$\pi_\theta(a \mid s)$	the policy	is literally the transformer + softmax head
τ	trajectory $(s_0, a_0, \ldots, s_T, a_T)$	a rollout is a batch of these
$p_\theta(\tau)$	trajectory distribution	only the π factor carries θ
$R(\tau)$	discounted return	has **no** θ ⇒ $\mathbb{E}[\nabla R] = 0$
R_t	reward-to-go from t	$\sum_{k\geq t}\gamma^{k-t}r_k$
$V^\pi(s)$	value: expected return from s	the standard baseline
$Q^\pi(s, a)$	action-value	"I don't care about the action" is V, not Q

$A^\pi(s,a)$	advantage $= Q^\pi - V^\pi$	"did this action help?" — what PPO optimises
$\hat{A}_t$	single-sample estimate of the advantage	in practice GAE, not a raw return
$b(s_t)$	baseline	must **not** depend on the action, or it biases the gradient
$r_t(\theta)$	importance ratio $\pi_\theta/\pi_{\text{old}}$	per-**token** in LLM implementations
ϵ	PPO clip range	typically 0.1–0.2
δ	TRPO trust-region radius	the KL bound PPO replaces with clipping
$r_\phi(x,y)$	a learned reward model	a **proxy** → reward hacking is its failure mode
π_{ref}	frozen reference policy (the SFT model)	the KL anchor; appears in both PPO and DPO
β	KL leash strength	small → chase reward; large → stay near π_{ref}
$Z(x)$	partition function	intractable — but **cancels** in DPO because both responses share x
$\hat{r}_\theta$	DPO implicit reward $\beta \log(\pi_\theta/\pi_{\text{ref}})$	DPO is not reward-free; the reward is implicit
λ	GAE decay	$\lambda \to 0$ low variance; $\lambda \to 1$ low bias

§14 Numbers and defaults worth remembering

quantity	typical	why
PPO clip ϵ	0.1–0.2	wide enough to learn, tight enough that the surrogate stays valid
$\lvert\mathcal{A}\rvert$ (action space)	$\sim 5\times10^4$–2×10^5	enormous for RL; makes on-policy sampling the bottleneck
models in memory, PPO	4	policy, reference, reward, value — plus optimiser states
models in memory, DPO	2	policy, reference
KL penalty β	10^{-2}–10^{-1}	below → reward hacking; above → the model never moves
DPO loss at init	$\ln 2 \approx 0.6931$	all log-ratios are 0 ⇒ gap 0 ⇒ $\sigma(0) = \frac{1}{2}$

§15 Worked example — one PPO term, one DPO step

15.1 The PPO clip, numerically

$$\text{vocab} = \{A, B, C\}, \quad \epsilon = 0.1, \quad \pi_{\text{old}}(A) = 0.50, \quad \pi_\theta(A) = 0.60$$

$$r = \frac{\pi_\theta(A)}{\pi_{\text{old}}(A)} = \frac{0.60}{0.50} = 1.20 \implies \text{clip}(r,\, 0.9,\, 1.1) = 1.10 \quad (r \text{ is above the band})$$

$\hat{A}$	unclipped $r\hat{A}$	clipped $1.1\hat{A}$	min	reading
$+0.8$	$1.20(0.8) = 0.96$	$1.10(0.8) = 0.88$	**0.88**	the clipped one wins → constant → **no gradient**. We already raised a good action's probability by more than ϵ; stop rewarding it.
-0.8	$1.20(-0.8) = -0.96$	$1.10(-0.8) = -0.88$	-0.96	the *unclipped* one wins → **gradient flows**. We wrongly raised a *bad* action's probability; the objective keeps pushing r down.

> **The asymmetry is the algorithm.** $\min$ makes the objective **flat** when you have moved too far in the *beneficial* direction, but keeps it **sloped** when you have moved too far in the *harmful* direction. Clipping never protects a bad update — it only refuses to reward a good one further. A $\max$ would invert this and let the policy run away.

15.2 One DPO step, numerically

With $\beta = 0.1$ and the model having drifted *toward the loser*:

	$\pi_{\text{ref}}(y \mid x)$	$\pi_\theta(y \mid x)$	$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$	$\hat{r}_\theta = \beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$
y_w	0.10	0.10	0	0
y_l	0.05	0.10	$+0.6931$	$+0.0693$

$$\text{gap} = \hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l) = -0.0693, \qquad \mathcal{L}_{\text{DPO}} = -\log\sigma(-0.0693) = -\log(0.4827) = 0.7283 \;>\; \ln 2$$

And after the model has learned:

	π_{ref}	π_θ	$\log \frac{\pi_\theta}{\pi_{\text{ref}}}$	$\hat{r}_\theta$
y_w	0.10	0.20	$+0.6931$	$+0.0693$
y_l	0.05	0.04	-0.2231	-0.0223

$$\text{gap} = +0.0916, \qquad \mathcal{L}_{\text{DPO}} = -\log \sigma(0.0916) = -\log(0.5229) = 0.6482 \;<\; \ln 2$$

Two checks you can run on any DPO implementation.

1. **At initialisation the loss is exactly** $\ln 2 \approx 0.6931$. When $\pi_\theta = \pi_{\text{ref}}$ every log-ratio is 0, so the gap is 0, so $\sigma(0) = \frac{1}{2}$. If your first step is not ≈ 0.693, your reference model is not a frozen copy of the policy — a classic bug.
2. **The gradient weight is** $\sigma\big(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\big) = 0.5173$ in the first case. That is the "surprise" factor from §10.5: the model currently ranks the loser above the winner, so it gets a large correction.

§16 Common confusions, one line each

the wrong statement	the correction
"PPO is on-policy."	**REINFORCE** is on-policy. PPO is **off-policy** via importance sampling — that is the entire reason the ratio exists.
"The advantage *is* the reward."	$A = Q - V$. It is the reward **minus a baseline**. A high-reward action in a high-value state can have zero advantage.
"A constant baseline reduces variance."	$\text{Var}(X - c) = \text{Var}(X)$. Only a **state-dependent** baseline correlated with the return helps (§5.2).
"Clipping keeps each update small."	Clipping flattens the *objective* outside the band; it does not bound the gradient. It is a cheap proxy for a KL constraint, not a step-size limit.
"The reward model gets better during PPO."	r_ϕ is **frozen**. Only π_θ and the value head move. Otherwise the reward would chase the policy.
"Higher reward means a better model."	It means a better **proxy** score. Past a point, reward up → quality down. That is reward hacking (§9.4).
"DPO is reward-free."	The reward is **implicit**: $\hat{r}_\theta = \beta \log(\pi_\theta / \pi_{\text{ref}})$. What DPO removes is the separate reward *network* and the rollouts.
"Larger β means stronger alignment."	Larger β means a **tighter leash to** π_{ref} — *less* optimisation. Too large and the model never changes.
"δ and ϵ are the same knob."	δ is a **KL radius** (TRPO). ϵ is a **probability-ratio band** (PPO). Related, not equivalent.
"The value head is part of the final model."	It is a **training-only** artefact for estimating the advantage. Discarded at inference.
"Reward scale matters."	$r \mapsto r + c$ is a symmetry of Bradley–Terry. Only **differences** are identified.
"DPO needs no preference data."	It needs **exactly** preference pairs and nothing else — no rollouts, no reward model. That is the trade.

§17 Where the pieces sit in a real pipeline

ADDED Neither source draws the full pipeline; the notes stop at "End of RLHF" on p146.

```
RLHF / PPO pipeline                    DPO pipeline
-------------------                    ------------
1. SFT on demonstrations  -> pi_SFT    1. SFT on demonstrations  -> pi_SFT
2. collect preference pairs (x,y_w,y_l)2. collect preference pairs (x,y_w,y_l)
3. fit r_phi by Bradley-Terry MLE      3. run L_DPO(theta) directly
4. PPO: maximise r_phi                 4. deploy pi_theta
   - beta*KL(pi_theta||pi_SFT)
   - with a value head for A_hat
5. deploy pi_theta
```

Two things to notice. First, π_{ref} **in both pipelines is** π_{SFT} — the KL leash is anchored to the supervised model, not to the pre-trained base. Second, **steps 1 and 2 are identical**; DPO and PPO differ only in what you do with the pairs. That is why DPO displaced PPO in most preference-alignment work: same data, two models instead of four, no rollouts. PPO (and its

GRPO descendants) survives where the reward is **verifiable** — math and code — because there you can skip step 2 entirely and compute r from a test harness.

§18 Appendix — the five derivations, stripped to exam length

Reproduce these from blank paper. Each line is annotated with the **reason** for the step, because that is what gets marked.

1. Policy gradient

$$\begin{aligned}
\nabla_\theta J &= \int \nabla_\theta p_\theta(\tau)\, R(\tau)\, d\tau && \text{gradient moves inside the integral}\\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau && \text{log-derivative trick}\\
&= \mathbb{E}_{\tau\sim p_\theta}\big[\nabla_\theta \log p_\theta(\tau)\, R(\tau)\big] && \text{back to an expectation}\\
\nabla_\theta \log p_\theta(\tau) &= \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) && \rho_0,\ p \text{ are } \theta\text{-free}
\end{aligned}$$

2. A baseline is free (needs b independent of the action)

$$\mathbb{E}_{a\sim\pi}\big[\nabla_\theta \log \pi_\theta(a \mid s) b(s)\big] = b(s) \sum_a \nabla_\theta \pi_\theta(a \mid s) = b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = b(s) \nabla_\theta 1 = 0$$

3. $R_t - V(s_t)$ estimates the advantage

$$\begin{aligned}
\mathbb{E}\big[R_t - V^\pi(s_t) \mid s_t{=}s, a_t{=}a\big] &= \mathbb{E}\big[R_t \mid s_t{=}s, a_t{=}a\big] - V^\pi(s) && \text{linearity; } V^\pi(s) \text{ fixed}\\
&= Q^\pi(s,a) - V^\pi(s) = A^\pi(s,a) && \text{definitions}
\end{aligned}$$

4. The surrogate matches the gradient at θ_{old}

$$\begin{aligned}
\nabla_\theta r_t(\theta) = \nabla_\theta \frac{\pi_\theta}{\pi_{\text{old}}} &= \frac{\pi_\theta}{\pi_{\text{old}}} \nabla_\theta \log \pi_\theta = r_t(\theta) \nabla_\theta \log \pi_\theta && \text{chain rule on a log}\\
\implies \nabla_\theta \mathbb{E}\big[r_t \hat{A}_t\big] &= \mathbb{E}\big[r_t \nabla_\theta \log \pi_\theta \hat{A}_t\big] = \nabla_\theta J && \text{at } r_t = 1, \text{ i.e. } \theta = \theta_{\text{old}}
\end{aligned}$$

5. DPO

(a) closed form: $\max_\pi \sum_y \pi(y)\Big[r(y) - \beta \log \frac{\pi(y)}{\pi_{\text{ref}}(y)}\Big]$ s.t. $\sum_y \pi(y) = 1$

$$\frac{\partial}{\partial \pi(y)} = 0 \implies \pi(y) \propto \pi_{\text{ref}}(y) \exp\big(r(y)/\beta\big) \qquad \text{the Lagrange multiplier is constant in } y \Rightarrow Z(x)$$

(b) invert: $\log \pi = \log \pi_{\text{ref}} - \log Z + r/\beta \implies r = \beta \log \frac{\pi}{\pi_{\text{ref}}} + \beta \log Z(x)$

(c) substitute: $r_w - r_l = \beta \log \frac{\pi(y_w)}{\pi_{\text{ref}}(y_w)} - \beta \log \frac{\pi(y_l)}{\pi_{\text{ref}}(y_l)} \underbrace{+\beta \log Z(x) - \beta \log Z(x)}_{=0}$ $Z(x)$ depends only on x; y_w, y_l share x

The four sentences that unlock all five.
1. Differentiate the *measure*, not the integrand — then convert with $\nabla p = p \nabla \log p$.
2. Any action-independent term has zero expected gradient, because $\sum_a \pi(a \mid s) = 1$.
3. $A = Q - V$ removes the state's own value, leaving only the action's contribution.
4. Two constants that depend only on x cannot change the *ranking* of two responses to that same x — so any such constant is invisible to Bradley–Terry.

§19 GRPO — the advantage without a reward model t=50:25, t=21:49

ADDED The lecture mentions GRPO twice — "GRPO is a version of TRPO where the advantage function is computed *without rewards*" t=50:25 — but p128–146 never introduces it. It is the one algorithm that closes the loop between §1.4 (verifiable rewards) and §7 (PPO), so it belongs here.

Recall the problem (§8): $\hat{A}_t$ needs a reward and a value function, and V_θ is an entire extra network. But if the reward is **verifiable** (§1.4) there is no reward model at all — the verifier *is* the reward function. And the baseline does not have to be a learned V; it only has to be something action-independent and correlated with the return (§5.2).

GRPO's move: sample a *group* of G completions for the same prompt, score each with the verifier, and use the group's own statistics as the baseline.

$$\mathcal{A}_i = \frac{r_i - \text{mean}\,(r_1, \ldots, r_G)}{\text{std}\,(r_1, \ldots, r_G)}, \qquad i = 1, \ldots, G$$

Each $\mathcal{A}_i$ replaces $\hat{A}_t$ in the *same* clipped objective of §7.8, with $\rho_{i,t} = \pi_\theta(y_{i,t} \mid x, y_{i,<t}) / \pi_{\theta_{\text{old}}}(\cdot)$ per token, plus the KL leash to π_{ref}.

	PPO / RLHF	GRPO	DPO
reward from	learned r_ϕ	**verifier** (test case, exact match)	preference pairs
baseline	learned V_θ	**group mean of G samples**	—
networks	4	2 (policy, ref)	2 (policy, ref)
needs rollouts	yes	yes, G per prompt	no
works for chat?	yes	**no** — needs a checkable r	yes

Why it matters. GRPO is the algorithm behind modern "reasoning" training on math and code: drop the reward model (the verifier is exact), drop the value head (the group is the baseline), and spend the compute on *more samples per prompt* instead. It is exactly the payoff of the observation in §1.4 — RLHF is strongest where the reward can be **checked**.

Not free. The normalisation by std makes the advantage scale-free, which is convenient but means a group where *every* sample scores identically (all pass, or all fail) has $\mathcal{A}_i = 0$ — the prompt contributes no gradient. Hard-but-solvable prompts carry the whole signal.

§20 Self-test

Part 1 — foundations (§1–§7). A. Why can't supervised fine-tuning do alignment? Name the format blocker and the locality blocker.
B. Define verifiable reward. Why do math and code dominate? What breaks in chat?
C. Write $p_\theta(\tau)$ and identify which factor carries θ. Why does that make ∇J hard?
D. Derive the policy gradient theorem in four lines. What does it free you from needing?
E. Prove $\mathbb{E}[R_t - V(s_t) \mid s_t, a_t] = A(s_t, a_t)$. Then why is $R_t - V(s_t)$ only an *estimator*?
F. Does subtracting a *constant* baseline reduce variance? One line of algebra.
G. Instantiate the MDP for an LLM: $s_t, a_t, \mathcal{A}, \rho_0, r$. When is r nonzero?
H. Why is the plain policy-gradient estimator on-policy, and what exactly does importance sampling buy? What does it cost?

Part 2 — PPO and DPO (§7–§10). I. Show $\nabla_\theta L = \nabla_\theta J$ at θ_{old}. Why the qualifier "at θ_{old}"?
J. Explain the PPO clip in both regimes of $\hat{A}_t$. Why $\min$ and not $\max$?
K. Write the full PPO loss. What is each of the three terms for? Which term is missing from p141?
L. Write the Bradley–Terry likelihood and its MLE objective. Which two properties are unmodelled?
M. Derive DPO: closed-form optimum, invert, substitute, cancel. *Exactly why* does $Z(x)$ cancel?
N. Is DPO reward-free? Answer precisely.
O. PPO needs 4 models, DPO needs 2. What does PPO buy for that cost?

Generated from `GenAI-IITM.pdf` p128–146 + `transript_RLHF.rtf`. Editable source: `viz/RLHF-Study-Notes.src.html`; build with `viz/build/build-notes.js`.